📚 Milestone 6 – Complete React Cheatsheet

⚛️ React কী?

React হলো একটি JavaScript লাইব্রেরি যা দিয়ে Web Application-এর User Interface (UI) তৈরি করা হয়।

কেন **React** শিখব?

- Reusable component বানানো যায়
- Virtual DOM ব্যবহার করে দ্রুত UI update
- Large application manageable করা যায়
- বিশাল ecosystem (npm packages)

React vs Angular vs Vue.js

- **Angular** → Complete framework (সব built-in)
- **Vue** → সহজ, lightweight
- **React** → শুধু UI library, বাকিটা তুমি পছন্দমতো যোগ করবে

🧩 Component

Component = UI-এর ছোট reusable অংশ।

যেমন:

- Navbar
- Card
- Button
- Footer

একবার বানালে বারবার ব্যবহার করা যায়।

দুই ধরনের:

- Functional Component (Modern)
- Class Component (Legacy)

```
function Hello() {
  return <h1>হ্যালো React!</h1>;
}
```

🔑 ছোট ছোট component = বড় app সহজে manage করা।

🧬 JSX (JavaScript XML)

JSX দেখতে HTML-এর মতো, কিন্তু আসলে JavaScript-এর ভিতরে লেখা UI।

পেছনে গিয়ে এটা React.createElement()-এ রূপান্তর হয়।

JSX Rules

- একটাই parent element
- {} এর ভেতরে JS expression
- camelCase attribute (onClick, className)
- inline style → object

```
const name = "রায়া";
return <h1>হ্যালো {name}</h1>;
```

📦 Props

Props = Parent → Child data flow

```
<Greeting name="রায়া" />

function Greeting({ name }) {
  return <h2>হ্যালো {name}</h2>;
}
```

গুরুত্বপূর্ণ বিষয়:

- Props read-only
- Child সরাসরি props পরিবর্তন করতে পারবে না
- Data সবসময় উপরে থেকে নিচে যায়

PropTypes

```
import PropTypes from "prop-types";

Greeting.propTypes = {
  name: PropTypes.string
};
```

Props ভুল টাইপে এলে warning দেখাবে।

🧵 Props Drilling

যখন Parent → Child → Grandchild → Great Grandchild
একই data বারবার পাঠাতে হয়, তখন props drilling হয়।

গভীর component tree হলে কোড messy লাগে।

সমাধান:

- Context API
- State management library

⬆️ Lifting State Up

যদি দুই sibling component একই state ব্যবহার করে:

✔ State parent-এ রাখো
✔ Props দিয়ে নিচে পাঠাও
✔ Callback দিয়ে parent update করো

এটাই React-এর clean data flow pattern।

🔄 State

State হলো component-এর নিজের data।

যখন:

- User click করে
- Input change হয়
- Data fetch হয়

তখন state বদলায় → React আবার render করে।

State dynamic UI-এর ভিত্তি।

🪝 Hooks

Hooks functional component-এ special ক্ষমতা যোগ করে।

---

useState

```
const [count, setCount] = useState(0);
```

- Local state তৈরি করে
- Immutable way-তে update করতে হবে

---

useEffect

Side effect handle করে:

- API call
- localStorage
- subscription

```
useEffect(() => {
```

```
  fetch('/api/data');
}, []);
```

Dependency array দিয়ে control করা যায়।

---

use()

Server Component-এ async data handle করতে ব্যবহৃত (Experimental concept)।

🔀 Conditional Rendering

React-এ UI conditionally দেখানো খুব সহজ।

- if/else
- ternary
- &&
- ||

```
{loggedIn ? <Profile /> : <Login />}
```

📋 List Rendering

Array render করতে map() ব্যবহার হয়।

```
{users.map(user => (
  <UserCard key={user.id} user={user} />
))}
```

🔑 Unique key দিতে হবে।

🖱️ Event Handling

React event camelCase-এ হয়।

```
<button onClick={handleClick}>Click</button>
```

Parameter পাঠাতে:

```
<button onClick={() => handleDelete(id)}>
```

Event handling মানে UI-কে interactive করা।

🔁 Rendering

Rendering মানে UI তৈরি হওয়া।

React:

1. Virtual DOM update করে
2. Diffing Algorithm দিয়ে পরিবর্তন খুঁজে বের করে
3. Real DOM-এ minimum change পাঠায়

Fiber engine এই process efficient করে।

⚙️ React Internals

Virtual DOM

Real DOM-এর lightweight copy।

Diffing Algorithm

Old vs New Virtual DOM তুলনা করে পার্থক্য বের করে।

Fiber

React-এর modern rendering engine।
Smooth ও interruptible rendering নিশ্চিত করে।

🌍 CSR vs SSR

CSR (Client-Side Rendering)

Browser JS load হওয়ার পর UI তৈরি হয়।

SSR (Server-Side Rendering)

Server আগে HTML বানিয়ে পাঠায় → দ্রুত initial load।

🌐 Data Fetching & Async

ডাটা আনার উপায়:

- fetch()
- axios
- useEffect
- use()

Best practice:

- Loading state রাখো
- Error state রাখো

🎨 Styling in React

1️⃣ CSS Import

```
import './App.css';
```

2️⃣ Inline Style

```
<div style={{ color: "red" }}>
```

3️⃣ Utility Framework

Tailwind CSS
DaisyUI

<div className={isActive ? 'bg-blue-500' : 'bg-gray-300'}>

📦 NPM & Packages

NPM = Node Package Manager

তুমি এখান থেকে package install করতে পারো।

উদাহরণ:

- axios
- react-icons
- chart library

package.json-এ:

- dependency list
- script
- version control

এটা পুরো project-এর configuration file।

🎯 Final Core Understanding

React আসলে ৫টা জিনিস ঘিরে:

1. Component thinking
2. Props & Data flow
3. State
4. Hooks
5. Efficient rendering

এইগুলো ভালোভাবে বুঝলে বড় প্রজেক্টও manageable হয়ে যায়।

React কঠিন না—ধাপে ধাপে layer খুললে সব পরিষ্কার।